Software Quality Engineering Solutions

Quiz 1: Control Flow Testing & Quiz 2: Data Flow Testing

QUIZ 1: CONTROL FLOW TESTING SOLUTIONS

Q1: Construct the Control Flow Graph (CFG)

Node Details (Line Numbers):

1. Start at `risk = 0`
2. `if income <= 0`
3. `return -1` (End)
4. `ratio = monthlyDebt / income`
5. `if ratio > 0.6`
6. `risk = risk + 40`
7. `risk = risk + 10`
8. `if creditScore < 300`
9. `return -2` (End)
10. `if creditScore < 580`
11. `risk = risk + 40`
12. `else if creditScore < 670`
13. `risk = risk + 25`
14. `else if creditScore < 740`
15. `risk = risk + 10`
16. `risk = risk + 5`
17. `if loanAmount > 100000`
18. `multiplier = 1.3`
19. `multiplier = 1.1`
20. `risk = risk * multiplier`
21. `if risk > 70 and ratio > 0.5`

22. `flag = "HIGH RISK"`
23. `flag = "MODERATE/LOW"`
24. `summary = risk + len(flag)`
25. `return summary` (End)

Edges (from → to):

- 1 → 2
- 2 → 3 (if true)
- 2 → 4 (if false)
- 4 → 5
- 5 → 6 (if true)
- 5 → 7 (if false)
- 6 → 8, 7 → 8
- 8 → 9 (if true)
- 8 → 10 (if false)
- 10 → 11 (if true)
- 10 → 12 (if false)
- 12 → 13 (if true)
- 12 → 14 (if false)
- 14 → 15 (if true)
- 14 → 16 (if false)
- 11 → 17, 13 → 17, 15 → 17, 16 → 17
- 17 → 18 (if true)
- 17 → 19 (if false)
- 18 → 20, 19 → 20
- 20 → 21
- 21 → 22 (if true)
- 21 → 23 (if false)
- 22 → 24, 23 → 24
- 24 → 25

Q2: Calculate Cyclomatic Complexity

Using formula: V(G) = E - N + 2P

- E = Number of edges = 27
- N = Number of nodes = 25
- P = Number of connected components = 1
- **V(G) = 27 - 25 + 2 × 1 = 4**

Alternative formula: V(G) = Number of decision nodes + 1 Decision nodes: 2, 5, 8, 10, 12, 14, 17, 21 = 8 decisions

V(G) = 8 + 1 = 9

Final Cyclomatic Complexity = 9

Q3: Basis Set — 9 Linearly Independent Paths

1. **Path 1:** 1→2→3 (income ≤ 0, early return -1)
2. **Path 2:** 1→2→4→5→6→8→9 (income>0, ratio>0.6, credit<300, return -2)
3. **Path 3:** 1→2→4→5→6→8→10→11→17→18→20→21→22→24→25 (high risk path)
4. **Path 4:** 1→2→4→5→7→8→10→11→17→18→20→21→22→24→25 (ratio≤0.6, credit<580, high risk)
5. **Path 5:** 1→2→4→5→7→8→10→12→13→17→18→20→21→23→24→25 (ratio≤0.6, credit 580–669)
6. **Path 6:** 1→2→4→5→7→8→10→12→14→15→17→18→20→21→23→24→25 (credit 670–739)
7. **Path 7:** 1→2→4→5→7→8→10→12→14→16→17→19→20→21→23→24→25 (credit ≥740, loan ≤100k)
8. **Path 8:** 1→2→4→5→7→8→10→12→14→16→17→18→20→21→23→24→25 (credit ≥740, loan >100k)
9. **Path 9:** 1→2→4→5→7→8→10→12→14→16→17→19→20→21→22→24→25 (risk>70 and ratio>0.5 true after else chain)

Q4: Identify At Least Two Infeasible Paths

Infeasible Path 1:

- **Path:** Income > 0, ratio > 0.6, creditScore < 300, continue to risk calculation
- **Why Infeasible:** Line 9 returns -2 if creditScore < 300, preventing further execution

Infeasible Path 2:

- **Path:** Income ≤ 0, then compute ratio and continue through credit check
- **Why Infeasible:** Line 3 returns -1 if income ≤ 0, exiting the function

Infeasible Path 3:

- **Path:** Risk ≤ 70 but ratio > 0.5, still take HIGH RISK branch
- **Why Infeasible:** Condition `risk > 70 and ratio > 0.5` requires both conditions true

---

Q5: Branch & Condition Coverage Design

Test	Input (income, creditScore, monthlyDebt, loanAmount)	Covered Branch	Why Necessary
1	(-100, 500, 200, 50000)	income <= 0 → return -1	Edge case: invalid income
2	(1000, 250, 500, 100000)	creditScore < 300 → return -2	Invalid credit score
3	(2000, 500, 1500, 50000)	ratio > 0.6, creditScore < 580, loanAmount ≤ 100k	High debt ratio, low credit
4	(2000, 600, 500, 50000)	ratio ≤ 0.6, creditScore 580–669	Moderate credit range
5	(2000, 700, 500, 50000)	creditScore 670–739	Good credit range
6	(2000, 800, 500, 50000)	creditScore ≥ 740	Excellent credit

Test	Input (income, creditScore, monthlyDebt, loanAmount)	Covered Branch	Why Necessary
7	(2000, 600, 500, 150000)	loanAmount > 100000	High loan amount multiplier
8	(2000, 800, 1200, 50000)	risk > 70 and ratio > 0.5 → HIGH RISK	Both risk conditions true
9	(5000, 800, 1000, 50000)	risk ≤ 70 → MODERATE/LOW	Low risk scenario

Coverage Verification:

- **Branch Coverage:** 100% achieved with above tests
- **Condition Coverage:** All boolean conditions tested with both True and False values

---

QUIZ 2: DATA FLOW TESTING SOLUTIONS

Q1: Definitions and Uses — per Variable

Variable	Definitions (Line #)	Uses (Line #)
risk	1, 6, 7, 11, 13, 15, 16, 20	6, 7, 11, 13, 15, 16, 20, 21, 25
ratio	4	5, 21
multiplier	18, 19	20
flag	22, 23	24
summary	24	25 (return)
income	Parameter	2, 4
creditScore	Parameter	8, 10, 12, 14
monthlyDebt	Parameter	4
loanAmount	Parameter	17

Q2: DU-chains for risk, ratio, and multiplier

Risk DU-chains:

1. (1 → 6), (1 → 7), (1 → 11), (1 → 13), (1 → 15), (1 → 16), (1 → 20)
2. (6 → 11), (6 → 13), (6 → 15), (6 → 16), (6 → 20)
3. (7 → 11), (7 → 13), (7 → 15), (7 → 16), (7 → 20)
4. (11 → 20), (13 → 20), (15 → 20), (16 → 20)
5. (20 → 25)

Ratio DU-chains:

1. (4 → 5)

2. (4 → 21)

Multiplier DU-chains:

1. (18 → 20)
2. (19 → 20)

Infeasible DU-paths:

- Any path where `creditScore < 300` (line 9) cannot reach later uses of risk/ratio
- If `income <= 0`, ratio definition never reaches any use

Q3: Data Flow Anomalies

Anomaly 1 — Potential OR (Definition never used)

- **Variable:** `multiplier`
- **Location:** Defined at lines 18/19, only used at line 20
- **Impact:** Could be optimized away, suggests possible design issue

Anomaly 2 — DD-like overlapping definitions without expressive artifact

- **Variable:** `risk`
- **Location:** Multiple redefinitions (lines 6,7,11,13,15,16,20)
- **Impact:** Overlapping definitions in different branches may cause maintenance confusion

Anomaly 3 — Possible precision / floating-point semantic anomaly

- **Location:** Line 4: `ratio = monthlyDebt / income`
- **Impact:** Floating-point rounding with large income/small debt affects `ratio > 0.5` check

Anomaly 4 — Semantic coupling of flag length into final score

- **Location:** Line 24: `summary = risk + len(flag)`
- **Impact:** Mixing numeric risk score with string length creates design anomaly

---

Q4: All-Defs, All-Uses, All DU-Paths — Test Sets

(1) All-Defs Coverage

Definitions to cover:

- risk: 1, 6, 7, 11, 13, 15, 16, 20
- ratio: 4
- multiplier: 18, 19
- flag: 22, 23
- summary: 24

Minimal test set (5 tests):

1. **T1:** (-100, 500, 200, 50000) → covers risk def at 1 only
2. **T2:** (2000, 500, 1500, 50000) → covers risk def at 6, ratio def, credit def at 11, multiplier def at 19, flag def at 23
3. **T3:** (2000, 600, 500, 150000) → covers risk def at 7, credit def at 13, multiplier def at 18
4. **T4:** (2000, 700, 500, 50000) → covers credit def at 15
5. **T5:** (2000, 800, 1200, 50000) → covers credit def at 16, flag def at 22

(2) All-Uses Coverage

Uses to cover:

- ratio used at 5, 21
- risk used at 6, 7, 11, 13, 15, 16, 20, 21, 25
- multiplier used at 20
- flag used at 24

Test set (reuse plus 2 extras):

- Tests T1-T5 from All-Defs
- **T6:** (2000, 800, 500, 50000) → covers risk > 70 false, ratio > 0.5 false
- **T7:** (2000, 600, 1200, 50000) → covers risk > 70 true, ratio > 0.5 true

(3) All DU-Paths Coverage

Complex DU-paths to cover:

- **A:** risk(1) → 6 → 11 → 20 → 25
- **B:** risk(1) → 7 → 13 → 20 → 25
- **C:** ratio(4) → 5 → later 21
- **D:** multiplier(18) → 20
- **E:** multiplier(19) → 20
- **F:** risk(1) → 16 → 20 → 21 → 25

Test inputs for DU-Paths:

1. (2000, 500, 1500, 50000) → Path A, C, E
2. (2000, 600, 500, 150000) → Path B, C, D
3. (2000, 800, 500, 50000) → Path F, C
4. (2000, 800, 1200, 50000) → Path F, C (ratio used at 21 true)

Q5: Branch & Condition Coverage Design

(Same as Quiz 1 Q5 above)

Q6: Loop-less Multi-branch Feasibility Trap

Example 1:

- **Attempted Path:** Income > 0, creditScore < 300, continue to loanAmount check
- **Infeasibility Reason:** Line 9 returns -2 immediately when creditScore < 300

Example 2:

- **Attempted Path:** Income ≤ 0, ratio > 0.6, proceed through credit checks
- **Infeasibility Reason:** Line 3 returns -1 immediately when income ≤ 0

Example 3:

- **Attempted Path:** Risk ≤ 70, ratio > 0.5, flag = "HIGH RISK"
- **Infeasibility Reason:** Condition `risk > 70 and ratio > 0.5` requires both true

---

SUMMARY

Both quizzes cover comprehensive white-box testing techniques:

- **Quiz 1** focuses on control flow analysis including CFG, cyclomatic complexity, basis paths, and branch coverage
- **Quiz 2** emphasizes data flow analysis including DU-chains, data flow anomalies, and various coverage criteria

The solutions demonstrate how to systematically analyze code structure to design effective test cases that uncover potential defects.

Solutions prepared based on pseudocode from Loan Eligibility & Risk Evaluation Module (LERM) Instructor: Natalia Chaudhry